# **OverPack**: Side-Loading Container Packing with a Robot-Aware Transformer Policy, a GPU-Parallel Digital Twin, and a 5-DOF Gantry Manipulator

Anush Bharathi R*

Technical report — draft of September 25, 2026

**Abstract**

Trucks and shipping containers are loaded through a single door: every item has to enter horizontally through one open face, and the load is built back-to-front and bottom-up. Most learned online 3D bin-packing policies assume top-down placement into an open bin, so they often choose placements that a robot working through the door cannot reach or cannot insert. We present **OverPack**, a small-scale side-loading packing system. It consists of a low-cost 5-DOF gantry manipulator with a twin electromagnet end effector (built and demonstrated in 2025), a GOPT-derived Transformer packing policy made robot-aware, and a GPU-parallel digital twin in Isaac Lab that sits between the policy and the open-loop robot.

The policy's key ingredient is a robot-aware candidate filter (RAP-CF). It computes exact insertion-corridor, end-effector-envelope and reachability masks for every placement of thousands of environments at once on the GPU. On the robot's box set, with the 4 mm lateral clearance that the twin shows is needed to avoid jams, the policy fills 45.9% of the container, against 38.2% for DBLF and 33.9% for best-fit. All 200 of its checked plans can be loaded in arrival order. A policy trained without RAP-CF packs as densely, but none of its plans can be loaded in arrival order. On the single recorded 19-box sequence of the 2025 robot, the policy only ties best-fit (50.9%). Over random orders of the same boxes it leads, with 50.7% against 46.5% for DBLF.

The exported plan executes in the twin with all 640 placements successful under domain randomization, all within 3.7 mm of target. Emulating the as-built firmware shows that it would execute that plan 7.2 times slower than planned and up to 135 mm off the planned path. A new firmware reproduces the planned trajectory at 1.00× duration within 0.2 mm. We also found that the surviving 2025 policy checkpoint was effectively untrained, because of an initialization bug. The original 2025 simulation and training code was lost, so everything here was produced with the open-source 2026 re-implementation described in this report.

## 1 Introduction

Loading cartons into trucks, trailers and shipping containers is still largely done by hand. The job is physically demanding, and how well it is done directly affects freight cost, since wasted volume costs money on every trip. Container loading has a long history in operations research, where much of the work concerns constraints that come up in practice: orientation, support, load-bearing

*Madras Institute of Technology, Anna University, Chennai, India. The OverPack robot was designed, built and demonstrated in 2025 as a B.E. capstone project by Anush Bharathi R, Kamran B, Nilavan K J and Shibi Praveen S under the guidance of Dr. P. Karthikeyan. The software stack described in this report (digital twin, packing environment, policy, planner and firmware bridge) is a 2026 re-implementation and extension; see Section 1.1.

strength, and the order in which items can be loaded and unloaded [1, 2]. A robot loading a container sees these constraints directly. It can only reach the load through the door, so every item has to be inserted *horizontally* through a single open face, and the stack grows from the back wall towards the door. Industrial truck-unloading systems [3] face the same geometry in reverse.

Learning-based online 3D bin packing has advanced quickly [4–8]. Most of this work, however, assumes an open-top bin filled from above by a suction gripper. In that setting a placement is feasible if it is inside the bin, does not overlap other items and is statically supported. With side-loading, feasibility also depends on two things:

1. **Insertion corridor.** The item, together with the end effector and arm that carry it, has to be able to slide in from the door to its target along a free corridor. A placement that is perfectly supported but sits behind an already-packed box is useless.
2. **Manipulator reachability.** The target pose, and every waypoint of the insertion, must be within the kinematic limits of the specific robot. On a compact gantry with a cantilevered arm, this removes a large part of the container.

A policy that ignores these conditions still reports high simulated utilization, but many of its plans cannot be executed. The mismatch does not show up until the plan reaches hardware, which is the most expensive place to find it. Low-cost robots make this worse. Our steppers run open loop without limit switches, the grasp is magnetic, and the parts are 3D-printed, so an execution error goes unnoticed until a box falls.

**OverPack.** This report describes OverPack, a small-scale side-loading packing system (Fig. 2). It has three parts (Fig. 1):

- **Hardware (built 2025).** A 5-DOF gantry manipulator on a $90 \times 60$ cm board: a dual-drive X axis, a Y carriage, a base turntable, a lead-screw Z lift, and a servo-driven head that yaws a pair of solenoid electromagnets. The robot picks paper-wrapped cardboard boxes by steel washers on one side face and loads them through the open face of a lidded $30 \times 20 \times 15$ cm acrylic container (Section 3).
- **A robot-aware packing policy.** The policy builds on GOPT's candidate-based Packing Transformer [8], with a placement generator that runs fully on the GPU. We add a multi-view depth observation, the RAP-CF robot-aware candidate filter (corridor, end-effector envelope and reachability masks), and the DSER shaped reward, whose geometric stability term can be checked against an Isaac Lab placement environment. It is trained with PPO [9] in a batched GPU environment (Section 4).
- **A digital twin and sim-to-real bridge.** This part includes an Isaac Lab 2.1 model of the robot and workcell built from the CAD-accurate geometry, Gymnasium environments, a motion planner that produces synchronized trapezoidal trajectories, and a compiler from trajectories to the exact serial protocol of the robot's Arduino firmware. A step-exact emulator of the as-built firmware re-integrates the compiled commands so that we can compare them with the planned trajectory (Section 5).

**Contributions.** (i) We formulate online packing under a side-loading constraint for a specific manipulator, and we give a mask construction (RAP-CF) that computes exact corridor, end-effector-envelope and reachability feasibility for every placement of every environment in a batch on the GPU. With it, a Transformer policy trained in a GPU environment (about 460 times the training

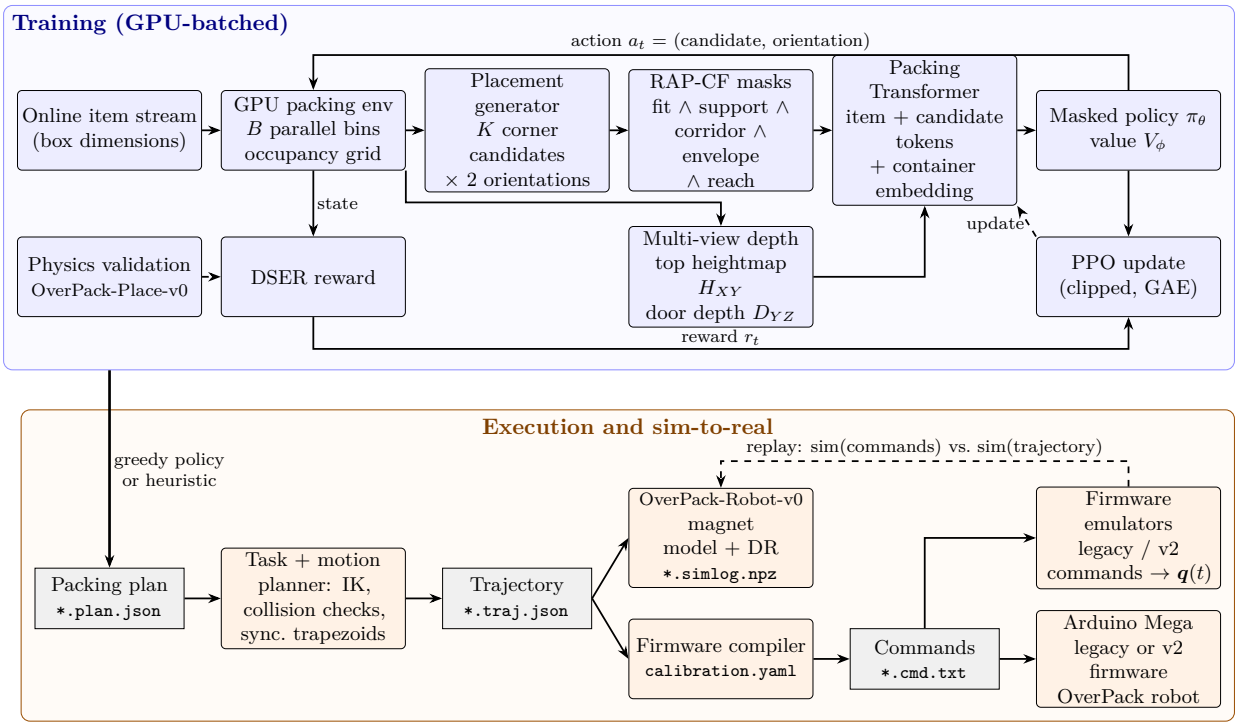



Figure 1: OverPack architecture. *Top:* the packing policy is trained in a batched GPU environment. Corner-point candidates are filtered by the robot-aware masks (RAP-CF), combined with a multi-view container embedding in a Packing Transformer, and optimized with PPO on the DSER reward. Stability is checked physically in a separate Isaac Lab placement environment (dashed: validation, not part of the reported training). *Bottom:* a plan, from the policy or a heuristic, becomes a synchronized trapezoidal joint trajectory. The trajectory runs in the Isaac Lab digital twin and is compiled into the firmware command stream. The firmware emulators re-integrate that stream so that the executed motion can be checked against the plan.

throughput of the 2025 code) packs 45.9% against 38.2% for the best heuristic, and every checked plan can be executed in arrival order. Without RAP-CF in training, no checked plan can be (Sections 6.2 and 6.3). (ii) We show in a physics twin why a side-loading plan needs clearance. Flush plans jam 22% of the time. Plans with a 4 mm clearance execute with 100% success under domain randomization (Section 6.6). (iii) We release a reproducible sim-to-real pipeline for a low-cost open-loop robot. The pipeline connects plan, trajectory, firmware command stream and simulation logs through documented file formats, so every stage can be replayed and checked in the digital twin. Emulating the as-built firmware exposed a timing mismatch that the planner could not see: the plan would run 7 times slower and up to 135 mm off-path. We provide a firmware that removes it. (iv) We document the as-built hardware as a reference design, identify its axis speeds from development footage, and report a negative finding about the surviving 2025 checkpoint.

## 1.1 Project history and provenance

OverPack grew out of an earlier entry by the lead author, PackMax/MaxPacker, for a truck-loading problem statement set by Godrej at Techfest, IIT Bombay (2021–22). That entry used heuristic 3D packing with a WebGL visualization. In 2025, the capstone team designed and built the robot described in Section 3 and demonstrated it running open loop. The pipeline was: a packing plan

from a GOPT policy trained with the authors' public code [8], a custom path planner, and relative step commands streamed to the Arduino. The simulation, path-planning and retrained-policy code of that final version was later lost together with the development laptop. What survived is the firmware, an earlier GOPT fork with one trained checkpoint, CAD files, photographs and development footage from 6 May 2025. Analysing that checkpoint showed that its actor had never learned, because of an initialization bug in the fork (Section 6.4). The software in this report is therefore a *re-implementation and extension* written in 2026 against the surviving hardware specification. Every quantitative result in this report comes from this code base. Numbers published by other authors are cited as theirs and are never reported as ours.

## 2 Related Work

**Heuristics for 3D packing and container loading.** Exact and heuristic methods for the three-dimensional bin-packing problem go back at least to Martello et al. [2]. Constructive heuristics differ mainly in how they list candidate positions. *Extreme points* (EP) [10] project the corners of packed items onto the container and onto other items. *Empty maximal spaces* (EMS) [11] keep the set of largest free cuboids, which may overlap. The *deepest-bottom-left-with-fill* (DBLF) rule [12] puts each item at the deepest, then lowest, then left-most feasible position. It matches back-to-front loading closely, which is why we use it as a baseline. Bortfeldt and Wäscher [1] survey the practical constraints of container loading, and Ali et al. [13] review offline and online approaches. Ha et al. [14] study an online container-loading heuristic under dynamic arrivals. These methods are fast and easy to interpret. However, their placement rules are written by hand and know nothing about the manipulator.

**Deep reinforcement learning for online 3D-BPP.** PackMan [4] learns to choose among heuristic-generated locations for a robotic loading arm. Zhao et al. [5] cast online 3D-BPP as a constrained MDP over a heightmap and learn a feasibility-mask predictor for stability. They extended this to practically feasible policies [15] and, with packing configuration trees (PCT) [6], to a tree-structured state over EMS/EP-style candidates that works in continuous domains. PackerBot [7] combines heuristics with DRL and a suction-based picking system for variable-sized products, and Yang et al. [16] integrate heuristics into DRL for online packing more generally. Attention-based encoders have also been applied to packing [17]. Closest to our work, GOPT [8] introduces a *Placement Generator* that turns the heightmap into a fixed-size set of EMS candidates, and a *Packing Transformer* that fuses item and candidate features by self- and cross-attention. Because the action space is defined over candidates rather than grid cells, GOPT generalizes across bin sizes. It was deployed on an industrial arm with a suction cup and overhead cameras, following the same authors' earlier reliable robot-packing system [18]. OverPack keeps GOPT's candidate-based action space and Transformer, and changes the environment and the mask: placements must be reachable through a single side opening by one specific low-cost manipulator.

**Physical realizability and stability.** Heuristic stability tests used in learning, such as support-area ratios or checking the center of mass against the support polygon, can disagree with real physics. Zhao et al. [19] learn physically realizable packing of irregular shapes using simulation. Wang and Hauser [20] and Wang and Hauser [21] plan stable, dense robotic packing of irregular objects with explicit stability reasoning. Recent work speeds up stability validation [22] and benchmarks online packing algorithms with robots in physics-based simulation [23]. We use a cheap geometric test

inside the mask and in the training reward, and we validate with physics separately: in a placement environment, and by replaying complete robot trajectories.

**Robotic packing systems.** JamPacker [24] combines an offline packing algorithm with a fault-recovery module for a vacuum-gripper arm, and PackerBot [7] and GOPT [8] demonstrate learned policies on industrial arms. These systems load from above. Truck-unloading robots [3] work through a trailer door, which is the same access geometry OverPack uses for loading. Our platform is much smaller and cheaper than any of these. That is a real limitation (Section 7), but it also makes the sim-to-real problems of open-loop hobby-grade hardware explicit.

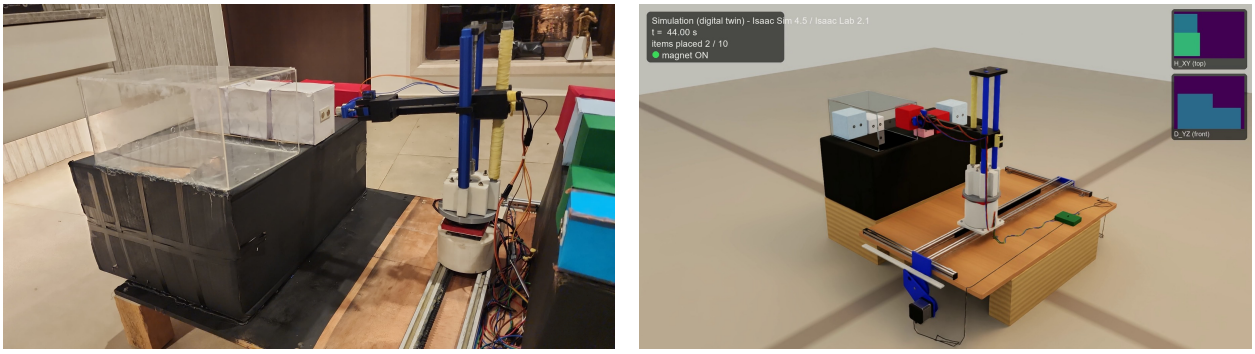
**Simulation for robot learning.** MuJoCo [25] made fast contact simulation common in robot learning. GPU-parallel simulators then moved physics and learning onto a single device: Isaac Gym [26], Orbit [27] and its successor Isaac Lab [28], and ManiSkill3 [29]. We use Isaac Lab 2.1 on Isaac Sim 4.5 [30] for the digital twin and photoreal rendering. The packing policy itself is trained in a lighter tensorized environment, because stepping full rigid-body physics for every candidate would dominate training time. For sim-to-real transfer we rely on domain randomization [31, 32] of physical and calibration parameters.

# 3 System

## 3.1 Mechanism

The robot is built on a $90 \times 60$ cm plywood board (Figs. 2 and 3). World frame $W$ has its origin on the board surface, with $x$ along the gantry rails and $z$ up. All dimensions below come from the single specification file `overpack/config/robot.yaml`. Each entry there is tagged with its provenance: bill of materials, project report, firmware, or measurement from photographs. The URDF, meshes, kinematics, planner and digital twin are all generated from this file.

- **X (dual drive).** A *bridge* of twin $20 \times 20$ mm aluminium extrusions, with a printed rack between them, rides on two 45 cm X rails fitted with printed racks. Each end is driven by a NEMA-23 stepper through a printed gear train (95 mm and 60 mm gears). The two motors face each other, so the firmware commands them with opposite signs. In the twin, the drives and downward-facing racks sit below the plywood, joined to the bridge by edge hangers. The hanger details and added floor clearance are documented design assumptions from the photos.
- **Y.** A cylindrical base carriage ($\varnothing 122 \times 100$ mm) travels along the bridge, driven by a pinion on the bridge rack.
- **Base yaw.** Inside the base, a stepper turns a turntable disc. Travel is limited in firmware to $\pm 1$ revolution to prevent the wires from twisting.
- **Z lift.** A six-lobed hub on the turntable carries three vertical columns and a centered T8 lead screw (8 mm lead). A printed nut-holder carriage rides on the columns and carries a horizontal two-bar arm, 0.30 m from the yaw axis to the servo axis.
- **Head yaw.** At the arm tip, an SG90 hobby servo yaws the end effector through $\pm 90°$.




(a) **Real robot**, 2025 photograph. (b) **Simulation** (Isaac Sim RTX), fixed view.

Figure 2: The OverPack workcell. A 5-DOF gantry manipulator loads washer-faced cardboard boxes from a pick row through the open face of a lidded acrylic container on a pedestal. (a) Photograph of the robot built in 2025. (b) The digital twin, generated from a single specification file and rendered by Isaac Sim from a fixed camera inspired by (a), during the policy's 10-box plan with the magnets on. The overlay in (b) is the simulator's own label.

Table 1: Actuated joints, $\boldsymbol{q} = [q_1, \ldots, q_5]$, in the order used throughout the software. Ranges come from the bill of materials, the report and the firmware. The stepper-axis velocity and acceleration limits were identified from the 2025 development footage (Table 11). The yaw value is a lower bound on the true peak speed, and the accelerations are conservative. The servo limits are design values.

| Joint | Type | Actuator / transmission | Range | $v_{\max}$ | $a_{\max}$ | Motor |
|---|---|---|---|---|---|---|
| $q_1$ `x_slide` | prism. | 2×NEMA-23, gear train + rack | $[0,\ 0.40]$ m | 0.17 m/s | 0.30 m/s$^2$ | 3, 4 |
| $q_2$ `y_slide` | prism. | stepper, pinion + rack | $[0,\ 0.56]$ m | 0.25 m/s | 0.40 m/s$^2$ | 5 |
| $q_3$ `base_yaw` | rev. | stepper, turntable | $[-2\pi,\ 2\pi]$ rad | 0.80 rad/s | 0.40 rad/s$^2$ | 1 |
| $q_4$ `z_lift` | prism. | stepper, T8 lead screw | $[0,\ 0.25]$ m | 0.045 m/s | 0.05 m/s$^2$ | 2 |
| $q_5$ `head_yaw` | rev. | SG90 servo | $[-\pi/2,\ \pi/2]$ rad | 5.0 rad/s | 40 rad/s$^2$ | servo |

**Kinematics.** The magnet frame $M$ sits midway between the two magnet pole faces, and its $+x$ axis points the way the poles face. Summing the fixed joint offsets in `robot.yaml` gives a constant $\boldsymbol{b} = (0.055,\ 0.060,\ 0.300)$ m. The forward kinematics are then

$$\boldsymbol{p}_M(\boldsymbol{q}) = \boldsymbol{b} + \begin{bmatrix} q_1 \\ q_2 \\ q_4 \end{bmatrix} + a \begin{bmatrix} \cos q_3 \\ \sin q_3 \\ 0 \end{bmatrix} + e \begin{bmatrix} \cos(q_3 + q_5) \\ \sin(q_3 + q_5) \\ 0 \end{bmatrix}, \qquad \psi_M = q_3 + q_5, \tag{1}$$

where $a = 0.300$ m is the arm length and $e = 0.032$ m is the distance from the servo axis to the pole faces. The task needs four coordinates, $(\boldsymbol{p}_M, \psi_M)$, and the robot has five joints, so there is one redundant degree of freedom. Fixing $q_3$ determines everything else in closed form: $q_5 = \psi_M - q_3$, and $q_1$, $q_2$ and $q_4$ follow linearly from (1). The redundancy is resolved by preferring $q_5 = 0$, which keeps the magnets aligned with the arm and the head inside the arm's swept width. When the slides cannot reach that solution, the solver minimizes $|q_5|$ over the feasible range of $q_3$. Because the feasible set is an arc of base positions clipped by the rectangular slide limits, the optimum is one of a handful of closed-form candidates, so the solver is exact and cheap enough to batch on the GPU (Sections 4.3 and 5.5). Both the picks and the insertions are done with the arm pointing towards $-x$ ($q_3 \approx \pi$).

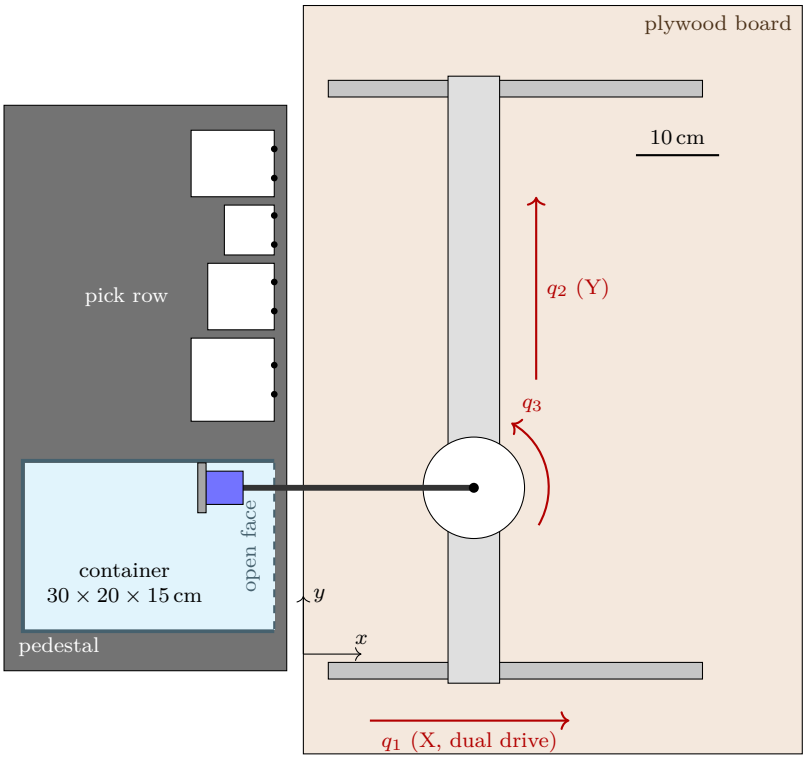



Figure 3: Top view of the workcell, drawn to scale from `robot.yaml`. The robot is shown at $q_1 = 0.15\,\text{m}$, $q_2 = 0.14\,\text{m}$, $q_3 = \pi$, i.e. with the arm reaching through the open face of the container. Pick-row boxes present their washer face (dots) in the plane of the container opening.

### 3.2 Electromagnetic end effector and boxes

The head carries two KK-P25/20 solenoid electromagnets (⌀25 × 20 mm, rated holding force 50 N each on ideal steel), 35 mm apart, with their poles facing forward. The boxes are paper-wrapped cardboard cubes and cuboids (6 cm, 8 cm and 10 cm cubes and 10 × 8 × 6 cm cuboids). Each box has two 12 mm steel washers on one side face, 35 mm apart at mid-height. The magnets pick a box by this face. They act on two thin washers rather than a thick steel plate, so the useful holding force is probably well below the rating. It has not been measured. The digital twin therefore treats it as a gap-dependent, randomized quantity (Section 5.3). An L298N dual H-bridge drives the magnets. The `pickup` command energizes them in attract polarity. The `place` command reverses the polarity to overcome remanent magnetization, holds it for 5 s and then switches off.

### 3.3 Workcell and task

The container is a lidded acrylic box with a 30 × 20 × 15 cm interior and 5 mm walls. It stands on a black pedestal next to the board, and its only opening faces $+x$, towards the robot. The lid makes top loading physically impossible. Boxes wait in a *pick row* on the same pedestal, with their washer faces flush with the container opening plane. In each cycle the robot approaches a pick-row box and engages the magnets. It then lifts and retracts, moves to the door, inserts the box horizontally along $-x$ to its target depth, lowers it onto its support, releases it and retracts. Boxes are loaded back-to-front and bottom-up, as in a truck. With the nominal geometry, the X travel is what limits how deep a box can be inserted: $q_1 = 0$ corresponds to the deepest insertion.

### 3.4 Electronics and firmware

An Arduino Mega 2560 controls five stepper drivers (DM542/TB6600 class), the servo and the magnet H-bridge. The firmware uses the AccelStepper/MultiStepper library [33]. It accepts newline-terminated ASCII commands over serial at 9600 baud (Table 12 in the appendix). The main command is `gantry,`$x$`,`$y$`,`$z$`,`$r$, a relative move in motor steps that all axes start together. There are no limit switches or encoders: `home` only resets the step counters, so each run starts from a manually aligned home pose and every later motion is open loop. This single fact shapes the whole sim-to-real design in Section 5. That section also describes a v2 firmware, written in 2026, which keeps this protocol and adds synchronized absolute moves.

## 4 Method: Robot-Aware Side-Loading Packing Policy

### 4.1 Side-loading online packing as an MDP

We discretize the container interior at resolution $\Delta = 0.5\,\text{cm}$ into an $X \times Y \times Z$ grid ($60 \times 40 \times 30$ for the physical container). The grid is expressed in the container frame $C$: $x$ is depth, measured from the back wall, with the open face at $x = X$; $y$ is width and $z$ is height. Items arrive one at a time with dimensions $\boldsymbol{d}_t = (l_t, w_t, h_t)$ drawn from a distribution $\mathcal{D}$. The agent must place the current item before the next one arrives. It also sees the dimensions of the next item (a one-item lookahead). It may not reorder items. During training it may not skip them either. On the fixed 19-box queue of Section 6, items that fit nowhere are skipped.

**Lateral clearance.** An open-loop robot cannot slide a box into a gap that has no tolerance. Every box therefore keeps a clearance of $\lambda = 4\,\text{mm}$ on each vertical side (`robot.yaml`). The environment packs *effective* footprints: the true box dimensions plus $2\lambda$, rounded up to the grid, i.e. $+1\,\text{cm}$. The true box is placed centred in its footprint. Utilization always counts the true volume. Section 6.6 shows why this is necessary: flush plans jam in the physics twin.

**State.** Each environment keeps two integer maps, which is all that the feasibility tests below need. The first is the top-down heightmap $\eta_t(x, y)$. The second is the *front surface*

$$\xi_t(y, z) = \max\big(\{\, x + 1 : \text{cell } (x, y, z) \text{ occupied} \,\} \cup \{0\}\big), \tag{2}$$

the depth of the front-most occupied cell in every $(y, z)$ row of the opening. The door depth map $D_t = (X - \xi_t)/X$ is what a depth camera looking into the open face would measure.

**Action.** The placement generator (Section 4.2) proposes $K$ candidate slots. Each slot is a footprint corner $(x_k, y_k)$ shared by both yaw orientations $o \in \{0, 1\}$, where $o = 1$ swaps $l$ and $w$. An action $a = oK + k$ places the item at $(x_k, y_k)$ with orientation $o$, resting on the heightmap at $z = \max \eta_t$ over its footprint. The action space therefore has $2K$ entries regardless of container size, as in GOPT.

**Termination and objective.** The episode ends when the current item has no feasible action under the mask of Section 4.3, or when the sequence is exhausted. With all dimensions in cells, the objective is the final space utilization $U = \sum_i l_i w_i h_i / (XYZ)$, reached only through placements that the robot can execute.

**Orientation in the physical cell.** The magnets always grip the $+x$ (door-side) face of an item. Choosing $o$ therefore means choosing which face is gripped, and the item is presented in the pick row in that orientation. In the physical cell the washers were fixed to one face, so presenting a cuboid in the other orientation requires preparing that face as well (Section 7).

### 4.2 GPU placement generator

GOPT's Placement Generator builds EMS candidates per environment in Python [8]. We replace it with a small number of batched tensor operations over all $B$ environments.

1. **Dense maps.** For every environment, orientation and footprint corner $(x, y)$ of the grid we compute: the resting height (a separable sliding-window maximum of $\eta_t$); bounds and lid clearance; the support terms of Section 4.3, from per-height integral images; and the three RAP-CF terms.
2. **Corner candidates.** From the valid placements we keep *corner / extreme points*: placements that cannot slide in $\pm x$ and in $\pm y$ without changing their resting height or becoming invalid. We add placements whose footprint corner coincides with a heightmap corner at the resting level, which is the EMS-style alignment of GOPT.
3. **Ranking.** Candidates are ranked in deepest-bottom-left order $(x, z, y)$, which is back-to-front loading. When more than $K$ remain, a max-pool non-maximum suppression spreads them out before the top $K$ are taken. The deepest-bottom-left valid placement is always a corner, so at least one slot exists whenever any valid placement exists.

Each slot is described by a 16-dimensional feature vector: its normalized position and local height, and for each orientation validity, resting height, top height, support ratio, supported-corner count and wall/neighbour contact. We use $K = 32$; on the physical box set, 99% of states have at most about 25 corner candidates before suppression.

### 4.3 RAP-CF: robot-aware candidate filtering

The mask over the $2K$ actions factorizes as

$$m(a) = m_{\text{fit}}(a) \wedge m_{\text{sup}}(a) \wedge m_{\text{cor}}(a) \wedge m_{\text{env}}(a) \wedge m_{\text{rch}}(a), \tag{3}$$

where the last three terms are the robot-aware part. All terms are evaluated densely in the batch before candidates are selected, so every one of the $K$ slots offered to the policy is feasible in at least one orientation. Infeasible logits are set to $-\infty$ before the softmax.

**Fit and support.** $m_{\text{fit}}$ requires the item to lie inside the container and below the lid. $m_{\text{sup}}$ is a conservative static-support test. At least half of the footprint must rest at the placement height, *and* either at least three of the four footprint corners must be supported, or the center of mass must lie inside the hull of the support cells. The hull test uses a sufficient condition that is cheap to evaluate: every closed quadrant around the center of mass contains a supported cell. Rules of this kind are standard in learned packing [5, 8], but they can disagree with physics, so we also measure stability physically (Sections 4.6 and 6.6).

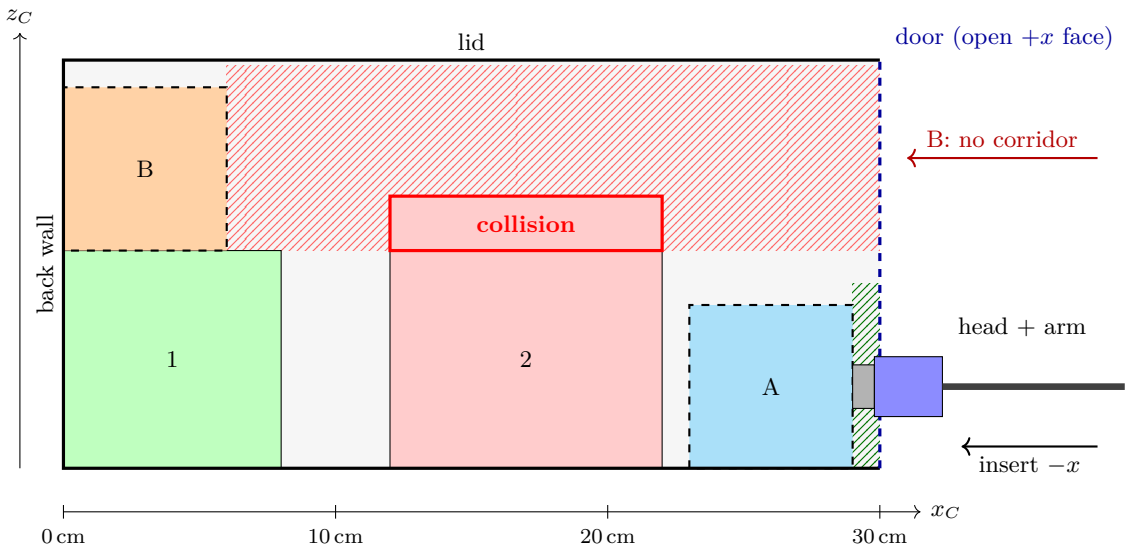



Figure 4: Side section through the container, showing the corridor term of RAP-CF. Candidate B rests on item 1 and passes every top-loading test (bounds, support). Its insertion corridor (hatched red, from its front face to the door, including the carry clearance $c$) intersects item 2, so $m_{\text{cor}} = 0$. Candidate A has a free corridor. In (4) this test reduces to a window maximum over the front surface $\xi_t$.

**Insertion corridor.** The item is pushed along $-x$ at height $z + c$ to depth $x$ and then lowered by $c$, so the prism $[x, X) \times [y, y + w) \times [z, z + h + c)$ must be free. From (2), this holds if and only if

$$m_{\text{cor}}(a) = \Big[ \max_{(y', z') \in [y,\, y+w) \times [z,\, z+h+c)} \xi_t(y', z') \le x \Big], \tag{4}$$

which is a separable sliding-window maximum over the $Y \times Z$ front surface. The carry clearance $c = 5\,\text{mm}$ is read from the same `robot.yaml` key that the task planner uses, so the policy's corridor model and the planner's are identical. For axis-aligned boxes the test is exact. Figure 4 shows why it matters: without it, a policy fills the back of an upper layer after it has already blocked the path there.

**End-effector envelope.** The magnet pair, servo housing and arm bars sweep a band of half-extents $(e_y, e_z)$ centred on the item's door-side face. From `robot.yaml`, $e_y = 30\,\text{mm}$ (the two $\varnothing 25\,\text{mm}$ magnets 35 mm apart) and $e_z = 25\,\text{mm}$ (the servo housing). This band travels from the item's front face $x + l$ out through the door. So $m_{\text{env}}$ requires the same window-maximum test over the band with threshold $x + l$, and it also requires the band to clear the side walls, the floor and the lid. For items narrower than the magnet pair, this term is what keeps the head from striking the neighbours of the insertion slot.

**Reachability.** $m_{\text{rch}}$ checks the inverse kinematics of the insertion pose: the magnet frame at the centre of the item's door-side face, facing $-x$. The inverse kinematics picks the solution with the smallest $|q_5|$ within all joint limits (Section 3). By default it also bounds $|q_5|$ so that the arm stays inside the envelope band. The workcell is fixed, so the result depends only on the face-centre cell. We precompute it once as a boolean table at half-cell resolution, and evaluating the mask during training is a gather.

### 4.4 Multi-view depth observation

GOPT's network sees the item and the candidate vectors but not the container itself [8]. For side loading, the layout of the load matters directly. Placing a tall item near the door early closes the corridor to everything behind it. We therefore encode two depth views: the top-down heightmap $\eta_t/Z$ ($X \times Y$, called $H_{XY}$) and the door depth map $D_t$ ($Y \times Z$, called $D_{YZ}$; the 2025 design called it $D_{XZ}$). Each view goes through a small convolutional encoder: three $3 \times 3$ convolutions with 32 channels, the first with stride 2 and the last one dilated, with normalized coordinate channels appended to the input. Global average and max pooling then give a view embedding. The two embeddings are fused by an MLP into a container embedding $\boldsymbol{e}_{\text{bin}}$, which is added to every candidate embedding and fed to the critic. In addition, the $H_{XY}$ feature map is sampled at each candidate's footprint corner, so each candidate also carries local context. The door view contains exactly the information the corridor mask is computed from, which lets the value function anticipate blockage before it happens.

The 2025 code already had a "multi-view processor". It applied a per-pixel MLP and then averaged over pixels, so it saw only the histogram of heights and not their spatial layout. That encoder is kept as an ablation option.

In the Isaac Lab placement environment (Section 5.2), the same two maps are rendered by simulated depth cameras above the lid and in front of the door. This gives a path towards camera-based observation.

### 4.5 Packing Transformer

The network follows GOPT's Packing Transformer [8], which builds on the standard Transformer [34]. *Item tokens* are the current item in both orientations plus the lookahead item. Each token holds its normalized dimensions, volume and presence flags, embedded by an MLP with learned type embeddings. *Candidate tokens* are the $K$ slot features, embedded by an MLP and summed with the container embedding and the local features of Section 4.4. Each of $N$ pre-norm blocks applies self-attention within the item tokens, self-attention within the candidate tokens (masking empty slots), candidate-to-item cross-attention and item-to-candidate cross-attention. The actor scores each action with a scaled bilinear product,

$$\ell(o,k) = \frac{\langle \phi_I(\boldsymbol{i}_o),\, \phi_C(\boldsymbol{c}_k) \rangle}{\sqrt{d}}, \qquad \pi_\theta(a \mid s) = \operatorname{softmax}_a\big(\ell(a) + \log m(a)\big), \tag{5}$$

with $\log 0 = -\infty$. The critic regresses $V_\phi(s)$ from the mean of the item tokens, the masked mean of the candidate tokens and $\boldsymbol{e}_{\text{bin}}$. We use $N = 4$ blocks, $d = 128$, 4 heads and a feed-forward expansion of 2, for 2.42 M parameters in total. For comparison, the 2025 configuration had 19.5 M.

### 4.6 DSER: Dynamic Stability-Enhanced Reward

The per-placement reward of the 2025 design is

$$r_t = \alpha \, \frac{l_t w_t h_t}{XYZ} + \beta \, R_t^{\text{den}} + \gamma \, R_t^{\text{sta}} + \delta \, R_t^{\text{div}}, \qquad (\alpha, \beta, \gamma, \delta) = (1,\, 0.05,\, 0.05,\, 0.01), \tag{6}$$

and every term lies in $[0, 1]$. The volume term is GOPT's reward; summed over an episode it equals the final utilization. $R_t^{\text{den}}$ is the fill fraction of the neighbourhood below the new item's top plane. The window is the footprint dilated by one cell, container walls count as filled, and any void trapped under the item counts as empty. $R_t^{\text{sta}} = \frac{1}{2}\rho_t + \frac{1}{2}\mu_t$ combines the support ratio $\rho_t$ with a centre-of-mass margin $\mu_t$: the distance from the footprint centre to the edge of the supporting cells' bounding

**Algorithm 1** One batched side-loading step (all operations over $B$ environments on the GPU).

**Require:** heightmaps $\eta \in \mathbb{R}^{B\times X\times Y}$, front surfaces $\xi \in \mathbb{R}^{B\times Y\times Z}$, current items $\boldsymbol{d} \in \mathbb{Z}^{B\times 3}$

**for** $o \in \{0, 1\}$ and every footprint corner $(x, y)$ **do**
  resting height $z \leftarrow$ window-max of $\eta$ over the footprint
  $m_{\text{fit}}, m_{\text{sup}}$ from bounds and per-level integral images of $\eta$
  $m_{\text{cor}}, m_{\text{env}} \leftarrow$ window-max tests on $\xi$ ▷ Eq. (4)
  $m_{\text{rch}} \leftarrow$ ReachTable[face centre]
**end for**
keep corner/extreme points; rank $(x, z, y)$; NMS; top-$K$ slots and features
$\ell \leftarrow$ PackingTransformer(items, slots, $\eta/Z, D$); $\ell[\neg m] \leftarrow -\infty$
sample $a \sim \text{softmax}(\ell)$; place item; update $\eta$ and $\xi$
$r \leftarrow$ DSER ▷ Eq. (6)
terminate and reset environments whose next item has an all-zero mask

box, normalized by the half-extent. $R_t^{\text{div}} = \min(\|(x_t, y_t)\|/10\,\text{cm}, 1)$ is the small "diversity" term of the 2025 design. It rewards placements away from the origin corner and saturates at 10 cm.

**Physics-validated stability.** The geometric $R^{\text{sta}}$ is a proxy, and all reported policies were trained with it. Two physics checks exist around it. First, the Isaac Lab placement environment `OverPack-Place-v0` (Section 5.2) simulates every placement and lets the load settle. It declares a placement stable only if the new box moves by at most 10 mm and tilts by at most 5°, and no previously placed box moves by more than 5 mm. Its reward is the volume term gated by this flag, minus penalties on the measured displacements and tilt. With the policy's candidate generator and DBLF actions, 97.8% of its placements are stable. The remaining ones are overhangs that the support heuristic accepts, and they are exactly the signal a physics-validated DSER would add. Second, the reward accepts an optional stability function that replaces $R^{\text{sta}}$ with a physics-measured score. Training or fine-tuning with that signal is not part of the results reported here. The policy's own plan is validated physically by full-trajectory replay instead (Section 6.6).

### 4.7 Training with PPO

The policy is trained with PPO [9]. We use the clipped surrogate objective, a clipped value loss, generalized advantage estimation [35] with $\gamma = 1$ as in GOPT, per-minibatch advantage normalization, and linearly decaying learning-rate and entropy coefficients. The action distribution is the masked categorical of (5). Environments, masks, observations, rollout storage and the network all stay on one GPU. A rollout is 16 batched `step` calls over $B = 2048$ environments, with automatic resets of finished episodes inside the step. Each rollout is followed by 4 epochs of minibatch updates in bf16. In contrast, the reference GOPT implementation runs CPU sub-process environments with Tianshou [36]. Item sequences are sampled on the fly. Evaluation uses fixed seeds and common random numbers, so every method sees the same sequences. Algorithm 1 summarizes one environment step, and Table 13 lists the hyperparameters.

## 5 Digital Twin and Sim-to-Real Bridge

The physical robot has no encoders, limit switches or cameras. Every plan therefore has to be proven executable *before* it is sent. The digital twin serves this purpose. It executes the same trajectory files, and the same compiled firmware command streams, that the real robot would




(a) Path-traced (64 spp): inserting a stacked box.

(b) RTX: the policy's 10-box plan, completed.

Figure 5: **Simulation (digital twin), rendered by Isaac Sim.** The insets show the $H_{XY}$ and $D_{YZ}$ maps of the load. The boxes are carried by the electromagnet force model.

receive. All artifacts are plain files with a documented schema: `*.plan.json` → `*.traj.json` → `*.cmd.txt`, plus `*.simlog.npz` from twin runs. Any stage can be replayed, diffed and visualized on its own.

### 5.1 Twin construction

**Geometry.** A parametric CAD script (CadQuery) builds as-built meshes for every link directly from `robot.yaml`. Each part is written in its link frame and tagged with a material from a fixed vocabulary (plywood, aluminium, colored PLA, acrylic, colored paper, steel), which Isaac Sim binds to physically based materials. Where the 2025 CAD survived, the meshes reuse it: the module-1.75 racks and the pinion come from the original STL files. Automated checks compare every exact-by-construction dimension of the meshes with `robot.yaml` to within 0.2 mm. The URDF has the seven links `board`, `bridge`, `base`, `turntable`, `carriage`, `head` and `magnet_tcp`, connected by the five joints of Table 1. It is also generated from `robot.yaml`. Collision geometry uses simple primitives, and `magnet_tcp` carries a thin box collider whose face is the pole plane. Isaac Lab's URDF converter imports the model, and a post-processing step flattens it into a single USD asset and binds the materials. A joint sweep in the simulator matches the kinematics module to within $4 \times 10^{-4}$ mm for every link.

**Physics.** The simulation runs at 200 Hz with 16 position iterations, enough for resting contact between light stacked boxes. Boxes are rigid bodies with mass from a cardboard density of 180 kg/m$^3$ plus two 4 g washers. Their colliders are convex hulls whose edges are chamfered by 1.5 mm to model the soft, paper-wrapped cardboard. The inner edges of the container's open face are chamfered by 2 mm. Friction coefficients come from `robot.yaml` (box–box 0.5, box–acrylic 0.35, magnet–washer 0.4). The joints are driven by stiff implicit PD drives that stand in for open-loop steppers. Gains are computed at start-up from the diagonal of the articulation mass matrix for a critically damped response with natural frequency 250 rad/s (slides and Z), 200 rad/s (turntable) or 120 rad/s (servo). The drives track the position and velocity targets of `traj.json` without re-profiling. The largest tracking error in a joint sweep is 0.16 mm (Z) and 1.2 mrad (servo).

### 5.2 Environments

We provide three Gymnasium [37] environments:

- `OverPack-Pack-v0`: the batched tensor packing environment of Section 4, used for training. It steps tens of thousands of placements per second (Table 7).
- `OverPack-Place-v0` (Isaac Lab): one step is one placement, chosen among the candidates of the policy's own generator, with the same grid, clearance and action layout. The robot is not simulated. Observations come either from analytic maps or from two $128 \times 96$ depth cameras per environment, one above the lid and one in front of the door. Depth points are back-projected, flying pixels and surfaces not facing the camera are removed, and each map cell takes the maximum of the remaining points, which gives $H_{XY}$ and $D_{YZ}$. The chosen box is placed 5 mm above its target (or pushed in along $-x$ by a virtual magnet) and the load settles before stability is measured (Section 4.6).
- `OverPack-Robot-v0` (Isaac Lab): executes a `traj.json` on the full twin. Joint targets are streamed at the physics rate, and the magnet model is switched on the `pickup`/`place` events. A feeder presents the next box in a pick-row slot once the previous one has left. The environment logs joint states, box poses and magnet state to `*.simlog.npz`. At the end it scores every item: success means the box ends within 10 mm of its planned position with a tilt of at most 5°. It also counts drops, where a washer separates while the magnet is on, and contacts of robot links with the container or pedestal above 1 N.

Videos and figures are rendered with Isaac Sim's own RTX renderer, in real-time or path-traced mode, from fixed camera presets. One uses the orientation recovered from the development footage, with a documented retreat to keep the full workcell in frame (Section 6.7). Every rendered frame carries a "simulation (digital twin)" label (Fig. 5).

**Visual wiring and acrylic.** Twelve visual-only Verlet/PBD cables follow clips expressed in link frames and collide with simple link proxies. Their routing, slack and mechanical coefficients are photo estimates or design choices; the cables exert no forces on the robot or boxes. The renderer updates USD curves at 30 Hz and checks stretch and attachment errors. For legacy RTX, exact ray refraction replaces image-based refraction, which otherwise projects a spurious black wedge over a foreground box. Acrylic shadows are disabled only in that renderer; path tracing retains normal transparent shadows. Neither correction changes the pedestal, container or box physics.

### 5.3 Electromagnet model

The simulator does not model magnetism, so the grip is applied as an external force. No kinematic attachment is used. For each washer $j$ we take the nearest pole face and compute the axial gap $g_j$ and the lateral offset $\rho_j$. The pair is *engaged* if the gap is below the 10 mm cutoff, the discs overlap, and the washer faces the pole to within 35°. While the magnets are on, an engaged washer is pulled towards the pole with

$$F_j = s \, \frac{f_{\max}}{1 + (g_j^+ / d_0)^2} \, \kappa(\rho_j), \tag{7}$$

where $f_{\max} = 50$ N and $d_0 = 1.5$ mm (`robot.yaml`), $g^+ = \max(g, 0)$, $\kappa$ is the overlap fraction of the pole and washer discs, and $s$ is a per-environment strength factor. The force acts at the washer, 1.2 mm proud of the box face. The total pull on one box is capped at $2 f_{\max}$, and an equal and opposite force acts on the head at the same points, so the magnet adds no net torque. The box is carried only by friction between the pole-face colliders and the box ($\mu = 0.4$) under this normal force. A box that is accelerated too hard, or gripped off-centre, therefore slips or drops as it would on the robot.

Table 2: Domain randomization in `OverPack-Robot-v0` (`configs/sim/sim.yaml`). Nominal physical values come from `robot.yaml`. The ranges are design choices, not measurements.

| Parameter | Randomization |
|---|---|
| Box dimensions (per axis) | $\pm 2$ mm |
| Box mass (nominal 180 kg/m$^3$ + washers) | $\times[0.8, 1.25]$ |
| Box friction coefficients | $\times[0.8, 1.2]$ |
| Magnet strength $s$ in (7) | $\times[0.7, 1.3]$ |
| Command latency (joint targets, magnet events) | [0, 50] ms |
| Trajectory time scale | $\times[0.97, 1.03]$ |
| Homing offset $(q_1, \ldots, q_5)$ | $\pm$(2 mm, 2 mm, 0.01 rad, 1 mm, 0.03 rad) |

Two modelling choices came from failures in the twin. First, *release*. The firmware reverses polarity, but soft steel is attracted by either polarity, so the default model lets the pull decay to zero over a 0.3 s pulse. A literal 2 N repulsive push slid 47–190 g boxes 5–24 mm past their targets, and only 4 of 7 boxes of a test plan landed within 10 mm. With the decaying model, all 7 landed within 0.4 mm. Second, *penetration*. A washer pressed against the pole face sits at a slightly negative gap. An early 2 mm tolerance released boxes that the head pushed into the back wall under randomized homing offsets. The tolerance is now 10 mm, so a box pressed against the poles stays held, as on a real magnet (Section 6.6). The value of $f_{\max}$ is the datasheet holding force on a full steel plate, and is probably optimistic for a 12 mm washer. Even the lowest randomized strength (35 N per washer) carries the heaviest box with a wide friction margin, and a pull-off measurement on the real washers is still needed (Section 7).

### 5.4 Domain randomization

In `OverPack-Robot-v0`, randomization covers the parameters that are uncertain on a hand-built robot (Table 2): hand-cut box dimensions and masses, friction, magnet strength, timing errors of the open-loop actuators, and homing offsets. The box-size draw is applied to the colliders and the washer positions alike. Following Tobin et al. [31] and Peng et al. [32], the goal is not to match the robot exactly but to measure how much margin a plan has against a plausible range of mismatch.

### 5.5 Task and motion planning

**Loading order.** A packing plan says *where* items go, in the policy's arrival order. For a robot that can only push items in through the door, that order may not be executable. The planner therefore builds a precedence graph. An item must be loaded after the items that support it. It must also be loaded before any item that would intersect its insertion corridor, where the corridor includes the magnet and servo cross-section and the carry clearance. A topological sort with the back-to-front, bottom-up key $(x, z, y)$ gives a loading order. A cycle, or a floating or overlapping box, marks the plan infeasible and names the offending items. We call a plan *arrival-order executable* if its own arrival order satisfies every precedence. This is the test used in Section 6.

**Waypoints.** For each item the task planner emits the fixed segment sequence `approach_pick`, `contact_pick`, `magnet_on`, `lift`, `retract_pick`, `transit`, `align_door`, `insert`, `lower`, `magnet_release` and `retract_door`, bracketed by `home`. The waypoint offsets are planner parameters: a 3 cm approach standoff, a 5 mm lift off the pedestal, the 5 mm carry clearance during insertion (then `lower`), and a 5 s release that matches the firmware's reverse-polarity hold.

Waypoints are magnet poses converted to joint space by the inverse kinematics of Section 3. That solver prefers $q_5 = 0$ and otherwise minimizes $|q_5|$ plus a small travel-time cost over the feasible range of $q_3$. It returns the representation of $q_3$ closest to the current one, so the turntable never unwinds needlessly. Pick, insertion and retraction are straight moves along the magnet axis, with the yaw joints fixed.

**Collision checking and path search.** The world model consists of axis-aligned boxes: the pedestal, the container panels and lid, waiting boxes and placed boxes. Each robot part is a box oriented by its yaw, attached to its link. Straight joint-space segments are checked by sampling them densely enough that no robot point moves more than 2 mm between samples, and inflating every clearance requirement by half that amount. Free-space motion needs a 2 mm margin. Intended contacts are allowed a 1.5 mm penetration tolerance: the held box sliding along walls and neighbours, and the magnet pair, which is exactly as wide as a 6 cm box. Between waypoints the planner tries, in order: the direct segment; structured detours (retract the bridge out of the workcell, lift to the top of the Z travel, rotate at the retracted pose); and RRT-Connect in joint space with a time-weighted metric and path shortcutting.

**Synchronized trapezoids.** Consecutive via points are joined by straight joint-space segments with a synchronized trapezoidal velocity profile [38]. Every segment starts and ends at rest, like consecutive `gantry` commands on the robot. The segment is parameterized as $\boldsymbol{q}(t) = \boldsymbol{q}_0 + s(t)\,\Delta\boldsymbol{q}$ with $s \in [0, 1]$. The per-axis limits (Table 1) bound $\dot{s} \le V = \min_i v_i/|\Delta q_i|$ and $|\ddot{s}| \le A = \min_i a_i/|\Delta q_i|$. The time-optimal profile under these bounds has duration

$$T = \begin{cases} 1/V + V/A & \text{if } V^2 \le A \quad \text{(trapezoid)}, \\ 2/\sqrt{A} & \text{otherwise} \quad \text{(triangle)}, \end{cases} \tag{8}$$

which is the exact minimum time of any synchronized straight-line move within the limits. Each axis follows a trapezoid proportional to its displacement. Trajectories are sampled at 10 ms into `*.traj.json` and validated independently (limits, collisions, events).

### 5.6 Firmware compiler, emulators and v2 firmware

**Calibration mapping.** Joint values map to firmware units as steps $= \sigma\,(q - q_{\text{home}})\,k$ for the four stepper joints. Here $\sigma$ is a sign, $q_{\text{home}}$ a home offset and $k$ the steps-per-unit constant. The servo command is $90^\circ + q_5$ in degrees. All constants live in `overpack/config/calibration.yaml`, each with a provenance status (Section 5.7).

**Compiler.** The compiler turns a trajectory into a command stream. Every waypoint is converted to *absolute* steps and rounded once. Each relative `gantry` command is the difference between consecutive rounded targets (in the firmware's $x, y, z, r$ argument order), so the commanded position stays within half a step of the plan at every waypoint, however long the plan. Rounding each increment separately would instead random-walk. Head-yaw targets become `1<deg>`, issued before the move so the servo slews concurrently, and magnet events become `pickup` and `place`. The legacy firmware parses each argument into a 16-bit `int`, so a move beyond $\pm 32767$ steps on one axis would wrap around silently. The compiler splits such moves. Under the earlier assumption of 1/8-step Z drive, any Z move over about 16 cm needed splitting. With the full-step Z inferred from the footage (Section 5.7), the whole stroke is 6250 steps, so the guard no longer triggers. An optional legacy

mode also sets each motor's maximum speed so that its own acceleration ramp lasts the planned segment time. This partly compensates for the timing issue described next.

**Legacy firmware emulator.** The emulator models the firmware exactly as it ran in 2025, and it exposed a mismatch with the planner's assumptions. `gantry` sets its targets through `MultiStepper::moveTo`, which computes speeds so that all axes finish together. The main loop, however, then calls each motor's `AccelStepper::run`, which recomputes that motor's own acceleration ramp up to its own maximum speed. The axes therefore start together but generally finish at different times, and the executed joint path is not the straight line the planner assumed. The emulator is a step-exact port of AccelStepper's timing recurrence. It also reproduces:

- the 16-bit argument wrap-around;
- `accel_off` being a no-op;
- `place` blocking the main loop for 5 s, which freezes the steppers and the serial port;
- `home` only zeroing the counters;
- stalls while status text drains through the 64-byte transmit buffer at 9600 baud.

Streaming a compiled command file through the emulator yields a joint trajectory that `OverPack-Robot-v0` can replay, so sim(*commands*) can be compared with sim(*planned trajectory*).

**v2 firmware.** To remove the mismatch we wrote a second firmware. It keeps every legacy command and adds an absolute `movej` command with 32-bit targets. The command carries the planned segment duration and is executed as a synchronized trapezoid: 10 ms motion segments, stepped by a 10 kHz Bresenham DDA in a timer interrupt, so step timing no longer depends on how busy the main loop is. The firmware also adds non-blocking magnet timing, soft limits, exactly one `DONE`/`ERR` acknowledgement per command, and 115200 baud. Its soft limits, speeds and step constants are compiled in from a header generated from `calibration.yaml`, so the firmware can no longer drift from the host calibration. A second emulator reproduces it tick by tick. The firmware source itself is compiled on the host against mocked Arduino and servo libraries, and tests check that it matches the emulator. A comparison tool reports the deviation between planned and emulated execution per joint: final error, error at every waypoint, deviation in time, deviation in normalized profile shape, and deviation from the planned straight path (Table 10).

## 5.7 Calibration

The steps-per-unit constants are derived from the mechanics:

- X: a simple gear train drives the rack. Every pitch circle moves at the same speed, so the travel per motor revolution is $\pi mz$ for the 32-tooth motor gear ($m = 1.75$ mm). At 1/8 step this gives 9095 steps/m.
- Y: the 20-tooth pinion gives 14551 steps/m at 1/8 step.
- Base yaw: direct drive gives 254.6 steps/rad at 1/8 step.
- Z: the 8 mm-lead screw at *full* step gives 25,000 steps/m.

The driver DIP-switch settings were never recorded, and the surviving 2025 scripts use four mutually inconsistent sets of constants. The microstep factors above are therefore inferences. For Z and yaw

they are the only settings consistent with the speeds measured in the footage and the firmware's default step rates (Section 6.7). For X and Y the footage fixes only the product of step setting and speed setting. No constant has been measured on the robot. `calibration.yaml` documents a measurement procedure for each axis: command a known step count, measure the travel with a rule, and repeat in both directions to expose backlash. The velocity and acceleration limits used by the planner (Table 1) come from the footage system identification.

# 6 Experiments

We ask six questions. (Q1) How well does the robot-aware policy pack, compared with heuristics, when every plan must be executable by the robot? (Q2) What does RAP-CF, and each other component, contribute? (Q3) What happened to the 2025 policy? (Q4) How fast is the pipeline? (Q5) Do the plans execute in the physics twin, including under randomization and through the firmware? (Q6) How does the twin compare with the only surviving footage of the real robot? Every number below comes from a run of this repository. The evaluation files are in `docs/media/policy_eval/`, `docs/sim.md`, `docs/footage.md` and `third_party/gopt_legacy/README.md`.

## 6.1 Setup

**Container and clearance.** All experiments use the physical container ($30 \times 20 \times 15$ cm). The main setting uses a 0.5 cm grid ($60 \times 40 \times 30$) and keeps a lateral clearance of 4 mm on every vertical side of every box. Footprints are padded by $2 \times 4$ mm, rounded up to the grid (so +1 cm), and each box is exported centred in its footprint. That leaves at least 4 mm to the walls and 8 mm between boxes. The clearance is not a free parameter. In the physics twin, flush-packed plans jam at the door, and this clearance removes the jams (Section 6.6). Where we also report a *flush* setting (1 cm grid, no clearance), it is labelled as such. Flush plans are not executable on the robot, so the flush numbers are for reference only.

**Item sequences.** **i.i.d.**: items drawn uniformly from the four physical box types, 2000 test episodes with a fixed seed and common random numbers across methods. **Physical queue**: the 19 boxes of the recorded 6 May 2025 session in their recorded pick order. Their total volume is 93.7% of the container, so items that cannot be placed are skipped. **Random orders**: the same 19 boxes in 1000 random orders.

**Metrics.** *Uti.* is the placed volume divided by the container volume (mean and 95% confidence interval). *#Items* is the number of items placed. *Arrival-order executability* measures whether the side-loading planner of Section 5.5 accepts the plan when boxes are loaded *in the order they arrive*, which is what an online loading robot must do. We check it on 200 plans per method. Every checked plan is also executable if the planner may reorder the items. That offline notion is weaker, and we report it only where it differs.

**Baselines.** All baselines act in the policy's own candidate set and mask. **DBLF** takes the lexicographic minimum of $(x, z, y)$: deepest, lowest, left-most. **Best-fit** minimizes the top height, then maximizes contact area, then falls back to DBLF order. **Random-feasible** samples uniformly among the masked actions. Each can be run with RAP-CF on (the default) or off.

Table 3: i.i.d. robot box set, 4 mm clearance, 2000 episodes. Arrival-order executability is checked on 200 plans. Decision time is per item for a single environment on the CPU; batched on the GPU, the policy needs 0.094 ms per item for 2000 environments.

| Method | Uti. (%) ↑ | Std | #Items | Arrival-order exec. | ms/item |
|---|---|---|---|---|---|
| **OverPack (greedy)** | **45.9** ± 0.4 | 8.5 | 7.90 | 200/200 | 27 |
| OverPack (sampled) | 45.7 ± 0.4 | 8.4 | 7.86 | – | – |
| OverPack, second seed (greedy) | 45.9 ± 0.4 | 8.4 | 7.88 | 200/200 | – |
| DBLF | 38.2 ± 0.3 | 8.0 | 6.71 | 200/200 | 12 |
| Best-fit | 33.9 ± 0.3 | 7.8 | 6.05 | – | 13 |
| Random-feasible | 22.6 ± 0.4 | 8.6 | 4.06 | 200/200 | – |

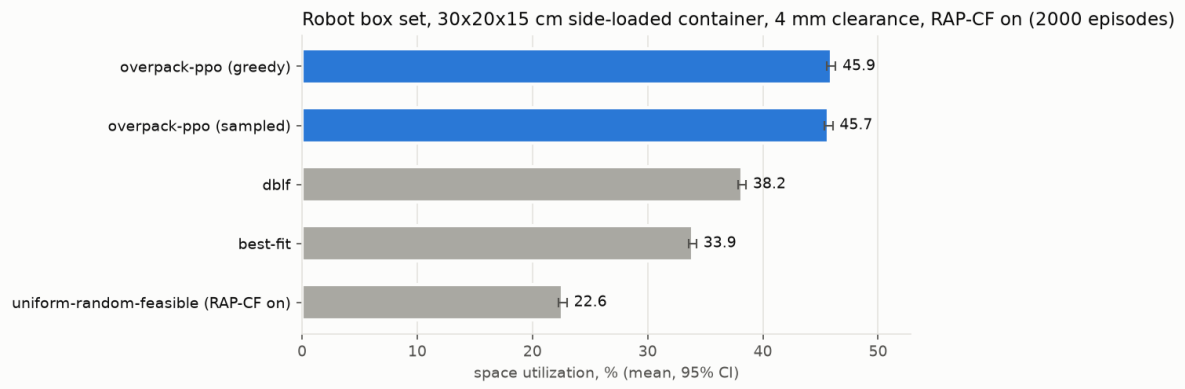



Figure 6: Space utilization on the i.i.d. robot box set (4 mm clearance, RAP-CF on, 2000 episodes, mean and 95% CI).

**Training.** The main policy has 2.42 M parameters. It was trained on the i.i.d. distribution for a fixed wall-clock budget: about $19.9 \times 10^6$ environment steps in 57 min on one RTX 4090, at about 6.4k steps/s end to end. A second seed and a no-RAP-CF variant had the same budget ($19.0 \times 10^6$ and $18.8 \times 10^6$ steps). Hyperparameters are in Table 13.

## 6.2 Packing performance (Q1)

Table 3 and Fig. 6 show the main result. With every plan executable in arrival order, the policy fills 45.9% of the container, against 38.2% for DBLF and 33.9% for best-fit. That is 7.7 points (20% relative) more than the best heuristic, and it places 1.2 more boxes per episode. The two seeds agree to within 0.1 point. Sampling instead of greedy decoding costs 0.2 points. Absolute utilization is low because the four box types are large relative to the container, and the 4 mm clearance takes 1 cm out of every footprint dimension. The policy's placements are also better supported than DBLF's (mean support ratio 0.984 against 0.960), on a par with best-fit (0.986).

**Physical queue.** Table 4 is less favourable. On the single recorded 19-box sequence, greedy decoding reaches 46.1% (seed 0) or 50.9% (seed 1). The plan we export picks the best of the greedy rollout and 256 sampled rollouts, and reaches 50.9% with 10 boxes. That *ties* best-fit (50.9%) and only slightly beats DBLF (50.0%). One fixed sequence is too small a test to separate the methods. Over 1000 random orders of the same 19 boxes, the policy averages 50.7% against 46.5% for DBLF and 40.3% for best-fit.

Table 4: The physical 19-box set (4 mm clearance, RAP-CF on, skipping allowed). Upper block: the recorded 2025 pick order. Lower block: 1000 random orders.

| Method | Placed / 19 | Uti. (%) | Arrival-order exec. |
|---|---|---|---|
| *Recorded 2025 order (one sequence)* | | | |
| OverPack greedy, seed 0 | 8 | 46.1 | yes |
| OverPack greedy, seed 1 | 10 | 50.9 | yes |
| OverPack sampled, 100 rollouts (mean) | 8.0 | 45.0 ± 0.9 | 100/100 |
| OverPack best of greedy + 256 samples (exported plan) | 10 | 50.9 | yes |
| DBLF | 11 | 50.0 | yes |
| Best-fit | 10 | 50.9 | yes |
| Random-feasible, 100 rollouts (mean / best) | 5.4 / 9 | 30.9 / 48.5 | 100/100 |
| *1000 random orders (mean ± 95% CI)* | | | |
| OverPack greedy | 10.65 | **50.7** ± 0.2 | – |
| DBLF | 10.70 | 46.5 ± 0.2 | – |
| Best-fit | 9.40 | 40.3 ± 0.3 | – |
| Random-feasible | 5.98 | 25.7 ± 0.5 | – |

Table 5: RAP-CF ablation (i.i.d. set, 4 mm clearance, 2000 episodes, 200 plans checked for executability). All checked plans remain executable when the planner may reorder the items.

| Variant | Uti. (%) | #Items | Arrival-order exec. |
|---|---|---|---|
| OverPack (RAP-CF in training and test) | 45.9 ± 0.4 | 7.90 | 200/200 |
| Trained without RAP-CF, evaluated without | 46.3 ± 0.4 | 7.98 | **0/200** |
| Trained without RAP-CF, RAP-CF at test only | 20.1 ± 0.2 | 3.48 | 200/200 |
| DBLF with / without RAP-CF | 38.2 / 38.6 | 6.71 / 6.81 | 200/200 / 148/200 |
| Random-feasible with / without RAP-CF | 22.6 / 32.3 | 4.06 / 5.81 | 200/200 / 47/200 |

**Flush setting (reference only).** Without clearance, on a 1 cm grid, a separately trained policy reaches 61.8% on the i.i.d. set against 52.2% (DBLF), 43.0% (best-fit) and 29.1% (random-feasible). On the recorded 19-box order it reaches 66.8% (13 boxes), which again ties best-fit (66.8%) and beats DBLF (62.0%). Over 1000 random orders it reaches 63.0% against 56.3% and 50.3%. The 16-point gap between the flush and clearance settings is the price of executability: 22% of flush-planned items jam in the physics twin (Table 8).

### 6.3 Ablations (Q2)

**RAP-CF must be part of training.** Table 5 isolates RAP-CF in the clearance setting, with the same training budget for every variant. A policy trained without RAP-CF packs 46.3%, as much as the full method. However, *none* of its 200 checked plans can be executed in arrival order. Every one needs items to be loaded in a different order from the one in which they arrive. Applying RAP-CF only at test time does not rescue it: utilization collapses to 20.1%, below random-feasible with RAP-CF, because the policy never learned to keep corridors open. The heuristics show the same effect in milder form. DBLF's back-to-front order already respects most corridors, but without the mask 26% of its plans are still not executable in arrival order. For random-feasible it is 77%.

**Other components (flush setting, unequal budgets).** The remaining ablations were run in the flush setting before the clearance requirement was known (Table 6). They ran under a short

Table 6: Component ablations in the *flush* setting (1 cm grid, no clearance; i.i.d. set, 2000 episodes, greedy). Budgets are unequal; see the text.

| Variant | Env. steps | Uti. (%) | Support ratio | $R^{\text{sta}}$ |
|---|---|---|---|---|
| Full method, seed 0 | $57.2 \cdot 10^6$ | 61.8 | 0.979 | 0.976 |
| Full method, seed 1 | $0.88 \cdot 10^6$ | 59.4 | 0.976 | 0.973 |
| −door view $D_{YZ}$ | $1.08 \cdot 10^6$ | 58.4 | 0.977 | 0.973 |
| 2025 view encoder (per-pixel MLP + mean) | $5.77 \cdot 10^6$ | 61.5 | 0.976 | 0.973 |
| −DSER (volume reward only) | $8.29 \cdot 10^6$ | 61.6 | 0.959 | 0.951 |
| DBLF / best-fit / random-feasible | – | 52.2 / 43.0 / 29.1 | – | – |

wall-clock budget on a shared GPU, so the step counts differ by up to 65×, and they must be read with that in mind. Two seeds of the full method differ by 2.4 points (61.8 vs. 59.4%), although the second seed also trained far fewer steps. The ablation differences in utilization lie within or near that spread. So we cannot claim a utilization effect for the door view, the 2025 view encoder or DSER. One effect is clear, however. Training on the volume reward alone reaches the same utilization with visibly less support: mean support ratio 0.959 against 0.976–0.979, and geometric stability score 0.951 against 0.973–0.976. That is the effect DSER's stability term is designed to have.

### 6.4 The 2025 checkpoint (Q3)

The only learned artifact that survived from 2025 is a GOPT checkpoint trained with the public GOPT code: EMS candidates, volume reward, a $20 \times 30 \times 15$ grid, and 50 epochs of $10^4$ environment steps at about 14 steps/s. We loaded it with the verbatim 2025 model code. It has 19.5 M parameters and loads with strict key matching. Analysing it gave a lesson rather than a baseline. *Its actor is input-independent.* Right after building the policy, the 2025 training script re-initializes every actor parameter from $\mathcal{N}(0, 0.01^2)$, and that includes the LayerNorm gains of the shared Transformer backbone. With gains near 0.01, the difference between candidates shrinks by about 10× per block: it is $2 \times 10^{-3}$ at the encoder and $4 \times 10^{-14}$ after eight blocks. The logits come out as the same constant for every action, equal to within one float32 ulp. The policy gradient is therefore essentially zero. The weights moved by only 2.3% between epochs 5 and 50, and the 2025 test reward stayed flat, so its "best" epoch is noise. On its own training distribution, the sampled policy scores $0.668 \pm 0.051$ and uniform random play scores $0.663 \pm 0.061$ (100 episodes each). In other words, the 2025 plans were uniform-random rollouts over GOPT's feasible set, and greedy decoding reduces to a deterministic bottom-back-left rule through tie-breaking.

On the physical queue, in its own flush protocol and with the corridor filter applied, argmax reaches 56.4% (10 boxes) and sampling $60.3 \pm 3.3$% (100 seeds). Without the corridor filter, 46 of 100 sampled plans cannot be loaded in arrival order. That is the side-loading failure that RAP-CF addresses. Because the protocols differ (flush, different support and pick-slot filters), these numbers are not directly comparable with Table 4. This finding is why we retrained from scratch instead of fine-tuning. None of GOPT's published results are involved. The failure is in our 2025 fork, not in GOPT [8].

### 6.5 Throughput (Q4)

Table 7 summarizes speed. Moving the whole loop onto the GPU gives end-to-end PPO throughput about 460 times that of the 2025 fork. At the same batch size the packing environment alone is

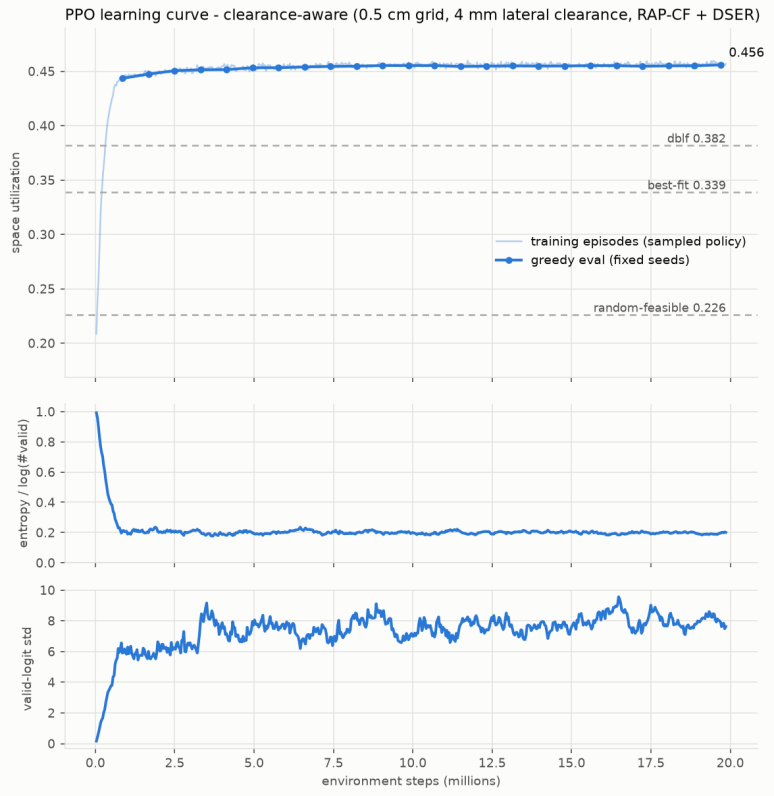



Figure 7: Training of the main (clearance-aware) policy. Top: utilization of training episodes and of the greedy evaluation on fixed seeds, with the heuristic baselines. Middle: normalized policy entropy. Bottom: spread of the valid logits. Utilization is within 1.5 points of its final value after about $10^6$ steps.

about seven times faster again, so the network and update now dominate training time. The Isaac Lab placement environment, with physically settled placements, runs at thousands of placements per second with analytic maps. With rendered depth it runs at about a thousand; 512 environments is the practical limit in 24 GB. Its rendered maps are accurate to a mean absolute error of 2.0 mm ($H_{XY}$) and 3.0 mm ($D_{YZ}$) against ground truth.

### 6.6 Execution in the digital twin (Q5)

**Clearance is necessary.** We replayed five planned trajectories in `OverPack-Robot-v0`: three flush-packed example plans and two plans with 3–10 mm gaps (Table 8). An item counts as *jammed* if, after settling, it is more than 10 mm or 5° from its target. In practice such boxes stick on the door edge or a neighbour 16–50 mm short, or they are knocked over. Of the flush items, 22% jam, whether box edges and the door are sharp or softened (1.5 mm edge radius, 2 mm door chamfer). The head also scrapes the container with up to 118 N. With clearance, nothing jams and there is no contact. Outcomes of individual flush plans flip with sub-millimetre contact changes, which is itself a reason not to plan flush on an open-loop robot.

**The policy's plan under randomization.** The exported 10-box plan for the physical queue was turned into a 158 s trajectory by the planner and replayed (Table 9). Nominally, all 10 boxes land within 1.05 mm of their targets with no tilt, drop or collision. Under full domain randomization

Table 7: Throughput on one RTX 4090 / i9-14900K. Environment steps are placements, except for `OverPack-Robot-v0`, where one step is 10 ms of simulated trajectory.

| Configuration | Parallel envs | Env. steps / s |
|---|---|---|
| Packing env only, clearance (0.5 cm grid) | 1k / 2k / 4k / 8k | 37.3k / 44.7k / 49.7k / 52.4k |
| Packing env only, flush (1 cm grid) | 1k / 2k / 4k / 8k | 77.4k / 117k / 157k / 191k |
| PPO end to end (env, policy, update), clearance | 2048 | 6.4–6.6k |
| PPO end to end, flush | 2048 | 8.1k |
| 2025 GOPT fork (CPU sub-processes, Tianshou) | 10 | 14 |
| Place-v0, analytic maps, stub generator | 1024 / 4096 | 5259 / 6221 |
| Place-v0, policy generator + DBLF actions | 256 | 2568 |
| Place-v0, two depth cameras, stub generator | 512 | 1059 |
| Robot-v0, full trajectory under DR | 64 | 1935 |

Table 8: Jam study in `OverPack-Robot-v0`: flush versus clearance plans. "Soft" adds a 1.5 mm box-edge radius and a 2 mm door chamfer.

| Plans | Items | Jam rate, sharp | Jam rate, soft | Head–container contacts |
|---|---|---|---|---|
| Flush, 0 mm gaps (3 plans) | 27 | 22% | 22% | 1–21 per plan, $\leq$118 N |
| Clearance, 3–10 mm gaps (2 plans) | 17 | 0% | 0% | none |

(Table 2), all 640 placements in 64 environments succeed (median error 1.52 mm, maximum 3.65 mm, no drops). The separately rendered 16-environment grid with seed 0 also placed all 160 boxes, with median error 1.67 mm, maximum 4.27 mm and no drops (Fig. 8). Five environments recorded robot contacts, with a peak force of 31.46 N; none disturbed a box. In the 64-environment run the head brushes the container in 12 environments (peak 70 N). This correlates weakly with the sampled base-yaw homing offset (Spearman $\rho = 0.33$).

**Randomization found bugs, not robustness limits.** The first randomized runs succeeded on only 78.8% of items in the 16-environment render, and on 83% of the first four items in a 64-environment run. We traced each failure to a cause, and none was a randomized physical parameter. The recording script stopped before the slowest environment had finished. The box-size randomization never reached the colliders, so the magnet model pulled at washers that sat inside the head. The magnet model's 2 mm penetration tolerance released a box that the head pressed into the back wall. After each was fixed, success was 100%. Enabling one randomization group at a time confirmed the diagnosis.

**Firmware fidelity.** Table 10 compiles the same trajectories for each firmware variant, emulates them, and compares the result with the plan. Quantization is the same for all three: at most 0.4 steps at the end and 0.47 steps at any waypoint, as the absolute-rounding compiler guarantees. Timing is not. The as-built firmware takes about seven times as long as planned, because every motor runs its own AccelStepper ramp. The magnet TCP then leaves the planned path by up to 135 mm (policy plan) or 142 mm (7-box plan). Setting per-motor speeds to match segment durations helps only partly (4.05×, 89 mm). The v2 firmware reproduces the planned trajectory at 1.00× duration, with at most 0.2 mm TCP deviation. Replayed in the twin, the v2 stream of the 7-box plan places 7/7 boxes within 0.4 mm. The v2 source is compiled on the host against hardware mocks, and its tick-level behaviour matches the Python emulator in tests. It has not yet run on the physical robot.

Table 9: Execution in `OverPack-Robot-v0`. Success: final position within 10 mm and tilt below 5° after a 1 s settle. "v2 firmware" replays the trajectory obtained by compiling the plan and emulating the v2 firmware.

| Plan | Condition | Placed | Pose err. (mm) median / max | Drops | Robot contacts (envs, peak force) |
|---|---|---|---|---|---|
| Policy plan (10 boxes) | nominal | 10/10 | 0.13 / 1.05 | 0 | none |
| | DR, 64 envs | 640/640 | 1.52 / 3.65 | 0 | 12/64, ≤70 N |
| | DR, 16 envs, rendered (seed 0) | 160/160 | 1.67 / 4.27 | 0 | 5/16, ≤31.46 N |
| 7-box plan (3–5 mm gaps) | nominal | 7/7 | – / 0.29 | 0 | none |
| | DR, 64 envs | 448/448 | 1.45 / 3.78 | 0 | none |
| | v2 firmware stream | 7/7 | – / 0.4 | – | – |

Table 10: Planned versus emulated execution. TCP deviation is the maximum distance between the emulated and planned magnet positions at equal normalized progress within each segment. Waypoint error is the maximum quantization error at any waypoint, in motor steps.

| Plan (planned duration) | Firmware (compiler mode) | Emulated duration | TCP dev. | Waypoint err. |
|---|---|---|---|---|
| Policy plan (158.0 s) | legacy (as is) | 1140 s (7.22×) | 135 mm | 0.47 |
| | legacy (duration-matched) | 640 s (4.05×) | 89 mm | 0.47 |
| | **v2** | 158.6 s (1.00×) | **0.2 mm** | 0.47 |
| 7-box plan (115.7 s) | legacy (as is) | 820 s (7.09×) | 142 mm | 0.46 |
| | **v2** | 116.1 s (1.00×) | **0.2 mm** | 0.46 |

**Cycle time.** The planned trajectory for the 10-box plan lasts 158.0 s, or 15.8 s per box (13.7–18.2 s per item). The firmware's 5 s reverse-polarity release accounts for 50 s of that, almost a third. All 91 motion segments were direct paths, with no detours or sampling-based planning needed. The trajectory has no collisions over 5329 swept samples, and its minimum clearance slack is 0.5 mm. The binding axes reach 100% of their velocity and acceleration limits, as expected of time-optimal synchronized profiles. The planner uses the axis speeds identified from the footage (Section 6.7).

### 6.7 Footage and simulation versus reality (Q6)

**What the footage is.** The only surviving video of the physical robot is a 172 s handheld compilation from one development session on 6 May 2025. It contains eight hard cuts, and the camera roll varies from −1° to 57°. We stabilize and level it using 35 hand-verified gravity keyframes. We match the camera of four shots to the robot geometry by line correspondences, with reprojection errors of 2.8–9.7 px. Phases of the two boxes carried in the clip are annotated semi-automatically.

**System identification.** Tracking the base, the Z carriage and the end effector in the calibrated shots gives plateau speeds of $0.25 \pm 0.03$ m/s (Y), $0.17 \pm 0.02$ m/s (X), $0.045 \pm 0.009$ m/s (Z) and at least 0.87 rad/s (base yaw) (Table 11). These are about 1.5 to 6 times the original design values, and they now set the planning limits in Table 1. Cross-checked against the firmware's default step rates, the Z speed is consistent only with a full-step driver (25,000 steps/m, not the 200,000 assumed earlier), and the yaw speed only with 1/8 microstepping. The X and Y speeds exceed what the defaults allow, so their speeds must have been changed during the session. From the footage we can identify only their product of microstepping and speed setting.




Figure 8: **Simulation (digital twin).** Sixteen domain-randomized environments on one GPU replaying the policy's 10-box plan (seed 0), at the end of the episode. Every environment placed 10/10 boxes. Tile labels give each environment's sampled magnet strength, latency and speed factor.

Table 11: Axis speeds identified from the 2025 development footage (video time, assuming real-time playback). $\sigma$ combines spread between moves, camera/model uncertainty and an 8% scale systematic.

| Joint | Measured plateau speed | Earlier design value | Moves used |
|---|---|---|---|
| $q_2$ `y_slide` | $0.25 \pm 0.03$ m/s | 0.040 m/s | 4 (0.21–0.33 m each) |
| $q_1$ `x_slide` | $0.17 \pm 0.02$ m/s | 0.060 m/s | 2 (0.07–0.16 m) |
| $q_4$ `z_lift` | $0.045 \pm 0.009$ m/s | 0.012 m/s | 2 (4–6 cm) |
| $q_3$ `base_yaw` | $\geq 0.87$ rad/s (lower bound) | 0.60 rad/s | 4 swings |

**Side-by-side.** Figure 9 pairs one calibrated shot (the second box, from approach to pick) with the twin rendered from a fixed camera using that shot's orientation and a retreat for full-workcell framing. The twin replays item 1 of the policy plan. The two are aligned phase by phase, not in time. In the clip the operator jogged the robot towards the box in many small moves over about 24 s, whereas the planned approach takes about 2 s, so the twin panel runs at 0.06–0.16 of real time. Two visible discrepancies are expected. The development layout of the pedestal and container differs from the final workcell in `robot.yaml`. The footage also shows only approach and pick; release is never visible. The comparison is therefore qualitative. It supports the geometry and camera model, not quantitative claims about execution.

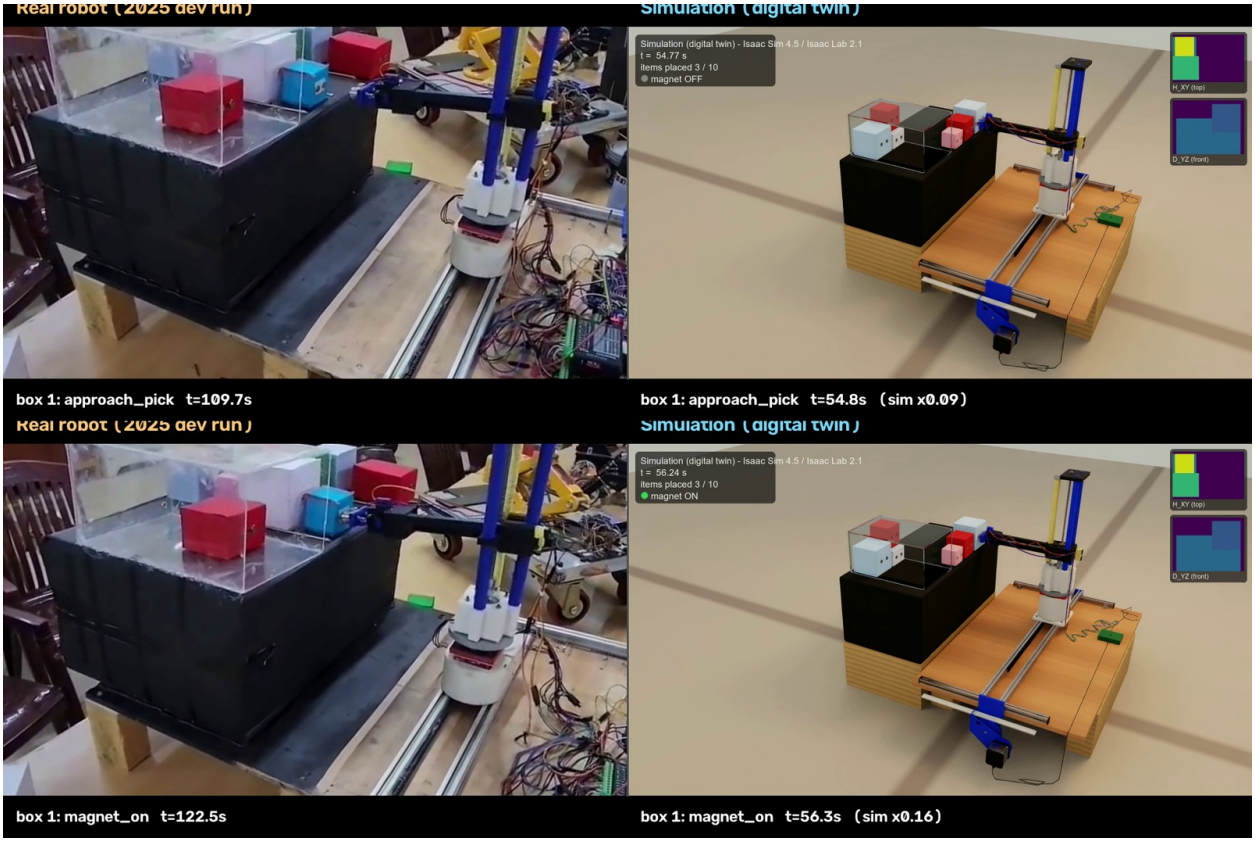



Figure 9: Left in each row: **real robot**, stabilized 2025 development footage. Right: **simulation (digital twin)**, rendered from a fixed camera based on that shot. The two are phase-synchronized (top: approach, bottom: magnet engaged). The sim panel's time warp is shown under it.

# 7 Limitations

**All quantitative results are from simulation.** The 2025 robot was built and demonstrated. The only surviving video, however, is handheld *development* footage from one lab session, edited into a compilation with eight cuts. The final demonstration was not recorded in a form we could recover. The sim-versus-real comparison is therefore qualitative, covers one box from approach to pick, and uses a workcell layout that differs from the final one (Section 6.7). Every quantitative result in this report comes from simulation with the 2026 re-implementation. None of the new plans, and neither firmware, has been run on the physical robot since.

**Open-loop execution.** The physical robot has no encoders, limit switches or cameras. Homing resets step counters around a manually aligned pose, and every motion after that is dead reckoning. Missed steps, backlash in the printed gear trains, and boxes that slip on the magnets are all undetectable at run time. The twin shows that a plan has margin *given* a calibration and a randomization range. It cannot correct errors while the robot is running. The depth maps the policy observes can be rendered by simulated cameras, but a camera-based closed loop has not been built on hardware.

**Uncalibrated physical parameters.** The magnet model's $f_{\max} = 50\,\mathrm{N}$ is a datasheet value for a full steel plate. The real pull on a 12 mm washer has not been measured, and it is probably lower. The twin's randomization range (±30%) may therefore overestimate it. No steps-per-unit constant has been measured. The microstep factors are inferred from the footage (Z, yaw) or remain ambiguous up to the product of microstepping and speed setting (X, Y), and the planner's axis speeds are footage estimates with 10–20% uncertainty (Sections 5.7 and 6.7). Before any live run, the driver DIP switches must be read and each axis measured.

**Strength of the packing evidence.** The i.i.d. results rest on 2000 episodes and two seeds. On the single recorded 19-box sequence, however, the policy only ties best-fit, and its greedy result depends on the seed (46.1% vs. 50.9%). The component ablations other than RAP-CF were run in the flush setting with unequal and short training budgets. They cannot resolve utilization differences smaller than the seed spread. All policies were trained with the geometric stability proxy of DSER. The physics-validated placement environment was used for validation, not for training.

**Scale and generality.** The workcell is a tabletop model. The container is $30 \times 20 \times 15\,\mathrm{cm}$, and there are four cardboard box types with steel washers on the gripped face. Grasping by magnet requires a ferromagnetic feature on every item. Orientation choices for cuboids require the operator to present the correct face, and the physical boxes carry washers on one face only. The policy sees the current item and one lookahead item. It cannot reorder items, and it does not choose the order in which items are picked. The candidate set ignores cavities under overhangs. The 4 mm clearance that makes plans executable costs about 16 points of utilization relative to flush packing. Real truck loading involves much larger and heavier items, load-bearing limits, weight distribution and unloading sequence constraints [1], none of which we model.

# 8 Conclusion

OverPack treats container packing as a problem of *side loading by a specific robot*, not of filling an open bin. A single-opening container adds feasibility conditions that top-loading policies do not see: a free insertion corridor, room for the end effector, and manipulator reachability. We compute all three exactly as batched masks (RAP-CF), and they have to be present during training. A policy trained with them fills 45.9% of the container against 38.2% for the best heuristic, with every checked plan executable in arrival order. A policy trained without them packs as densely, but none of its plans can be executed that way. Around the policy, a GPU-parallel digital twin and a firmware-exact compile-and-emulate path connect every plan to the command stream a low-cost open-loop robot would execute. That chain has already paid off three times. It showed that flush plans jam and that a few millimetres of clearance remove the jams. It showed that the as-built firmware would run the plan seven times slower and far off its path, which simulating the planned trajectory alone would never have revealed. And it showed that the surviving 2025 checkpoint had never learned.

Future work follows from the limitations. The first step is to measure the robot: steps-per-unit, the magnet pull on a washer, and a first run of the v2 firmware. The next is to close the loop with a door-mounted depth camera, reusing the view already modeled in the policy and the twin. Training with the physics-validated stability signal, item-order selection, and a larger workcell closer to logistics practice come after that.

## Acknowledgements

The OverPack robot was designed, built and demonstrated in 2025 as a B.E. capstone project at Madras Institute of Technology, Anna University, by Anush Bharathi R, Kamran B, Nilavan K J and Shibi Praveen S. We thank our project guide, Dr. P. Karthikeyan, for his guidance and support throughout the project. The packing policy builds on the open-source implementation of GOPT by Xiong et al. [8]. The digital twin is built on NVIDIA Isaac Sim and Isaac Lab. The problem was first posed to the lead author through the Godrej truck-loading problem statement at Techfest, IIT Bombay (2021–22).

## References

[1] Andreas Bortfeldt and Gerhard Wäscher. Constraints in container loading – a state-of-the-art review. *European Journal of Operational Research*, 229(1):1–20, 2013. doi: 10.1016/j.ejor.2012.12.006.

[2] Silvano Martello, David Pisinger, and Daniele Vigo. The three-dimensional bin packing problem. *Operations Research*, 48(2):256–267, 2000.

[3] Paul Doliotis, Christopher D. McMurrough, Alex Criswell, Matthew B. Middleton, and Samarth T. Rajan. A 3D perception-based robotic manipulation system for automated truck unloading. In *IEEE International Conference on Automation Science and Engineering (CASE)*, pages 262–267, 2016. doi: 10.1109/COASE.2016.7743416.

[4] Richa Verma, Aniruddha Singhal, Harshad Khadilkar, Ansuma Basumatary, Siddharth Nayak, Harsh Vardhan Singh, Swagat Kumar, and Rajesh Sinha. A generalized reinforcement learning algorithm for online 3D bin-packing. arXiv:2007.00463. AAAI 2020 Workshop on Generalization in Planning, 2020.

[5] Hang Zhao, Qijin She, Chenyang Zhu, Yin Yang, and Kai Xu. Online 3D bin packing with constrained deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 741–749, 2021.

[6] Hang Zhao, Yang Yu, and Kai Xu. Learning efficient online 3D bin packing on packing configuration trees. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://openreview.net/forum?id=bfuGjlCwAq.

[7] Z. Yang, S. Yang, S. Song, W. Zhang, R. Song, J. Cheng, and Y. Li. PackerBot: Variable-sized product packing with heuristic deep reinforcement learning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5002–5008, 2021.

[8] Heng Xiong, Changrong Guo, Jian Peng, Kai Ding, Wenjie Chen, Xuchong Qiu, Long Bai, and Jianfeng Xu. GOPT: Generalizable online 3D bin packing via transformer-based deep reinforcement learning. *IEEE Robotics and Automation Letters*, 9(11):10335–10342, 2024. doi: 10.1109/LRA.2024.3468161.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[10] Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. Extreme point-based heuristics for three-dimensional bin packing. *INFORMS Journal on Computing*, 20(3):368–384, 2008.

[11] Francisco Parreño, Ramón Alvarez-Valdés, José M. Tamarit, and José F. Oliveira. A maximal-space algorithm for the container loading problem. *INFORMS Journal on Computing*, 20(3): 412–422, 2008.

[12] Korhan Karabulut and Mustafa Murat İnceoğlu. A hybrid genetic algorithm for packing in 3D with deepest bottom left with fill method. In *Advances in Information Systems (ADVIS 2004)*, volume 3261 of *Lecture Notes in Computer Science*, pages 441–450. Springer, 2004. doi: 10.1007/978-3-540-30198-1_45.

[13] Sara Ali, António Galrão Ramos, Maria Antónia Carravilla, and José Fernando Oliveira. On-line three-dimensional packing problems: A review of off-line and on-line solution approaches. *Computers & Industrial Engineering*, 168:108122, 2022. doi: 10.1016/j.cie.2022.108122.

[14] Chi Trung Ha, Trung Thanh Nguyen, Lam Thu Bui, and Ran Wang. An online packing heuristic for the three-dimensional container loading problem in dynamic environments and the physical internet. In *Applications of Evolutionary Computation (EvoApplications 2017)*, volume 10200 of *Lecture Notes in Computer Science*, pages 140–155. Springer, 2017. doi: 10.1007/978-3-319-55792-2_10.

[15] Hang Zhao, Chenyang Zhu, Xin Xu, Hui Huang, and Kai Xu. Learning practically feasible policies for online 3D bin packing. *Science China Information Sciences*, 65(1):112105, 2022. doi: 10.1007/s11432-021-3348-6.

[16] S. Yang, S. Song, S. Chu, R. Song, J. Cheng, Y. Li, and W. Zhang. Heuristics integrated deep reinforcement learning for online 3D bin packing. *IEEE Transactions on Automation Science and Engineering*, 21:939–950, 2024.

[17] Q. Que, F. Yang, and D. Zhang. Solving 3D packing problem using Transformer network and reinforcement learning. *Expert Systems with Applications*, 214:119153, 2023.

[18] Heng Xiong, Kai Ding, W. Ding, Jian Peng, and Jianfeng Xu. Towards reliable robot packing system based on deep reinforcement learning. *Advanced Engineering Informatics*, 57:102028, 2023.

[19] Hang Zhao, Zherong Pan, Yang Yu, and Kai Xu. Learning physically realizable skills for online packing of general 3D shapes. *ACM Transactions on Graphics*, 42(5):1–21, 2023. doi: 10.1145/3603544.

[20] Fan Wang and Kris Hauser. Stable bin packing of non-convex 3D objects with a robot manipulator. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8698–8704, 2019.

[21] Fan Wang and Kris Hauser. Dense robotic packing of irregular and novel 3D objects. *IEEE Transactions on Robotics*, 38(2):1160–1173, 2022. doi: 10.1109/TRO.2021.3097261.

[22] Ziyan Gao, Lijun Wang, Yuntao Kong, and Nak Young Chong. Online 3D bin packing with fast stability validation and stable rearrangement planning. arXiv:2507.09123, 2025.

[23] Zhoufeng Wang, Hang Zhao, Juzhan Xu, Shishun Zhang, Ruizhen Hu, Chenyang Zhu, Zecui Zeng, Weiyan Zhu, Zeyu Xiong, Haibin Yu, and Kai Xu. RoboBPP: Benchmarking robotic online bin packing with physics-based simulation. arXiv:2512.04415, 2025.

[24] M. Agarwal, S. Biswas, C. Sarkar, S. Paul, and H. S. Paul. Jampacker: An efficient and reliable robotic bin packing system for cuboid objects. *IEEE Robotics and Automation Letters*, 6(2): 319–326, 2021.

[25] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

[26] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and Gavriel State. Isaac Gym: High performance GPU-based physics simulation for robot learning. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[27] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. doi: 10.1109/LRA.2023.3270034.

[28] Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, Octi Zhang, Peter Du, et al. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. arXiv:2511.04831, 2025.

[29] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, et al. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI. arXiv:2410.00425, 2024.

[30] NVIDIA. Isaac Sim. https://developer.nvidia.com/isaac/sim, 2025. Version 4.5.

[31] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017. doi: 10.1109/IROS.2017.8202133.

[32] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018. doi: 10.1109/ICRA.2018.8460528.

[33] Mike McCauley. AccelStepper: Arduino library for stepper motors. https://www.airspayce.com/mikem/arduino/AccelStepper/, 2026. Includes the MultiStepper class. Accessed 2026-09-23.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[35] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[36] Jiayi Weng, Huayu Chen, Dong Yan, Kaichao You, Alexis Duburcq, Minghao Zhang, Yi Su, Hang Su, and Jun Zhu. Tianshou: A highly modularized deep reinforcement learning library. *Journal of Machine Learning Research*, 23(267):1–6, 2022.

[37] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032, 2024.

[38] Luigi Biagiotti and Claudio Melchiorri. *Trajectory Planning for Automatic Machines and Robots*. Springer, Berlin, Heidelberg, 2008. doi: 10.1007/978-3-540-85629-0.

# A Firmware command protocol

Table 12: Serial protocol of the as-built Arduino firmware (9600 baud, newline-terminated ASCII). The compiler emits only the commands in the upper block. The lower block is used for manual bring-up. The v2 firmware (Section 5.6) accepts all of these plus `movej` and acknowledgements at 115200 baud.

| Command | Effect |
|---|---|
| `gantry,`$x$`,`$y$`,`$z$`,`$r$ | Relative move in motor steps for X (motors 3 and 4, opposite signs), Y (5), Z lift (2) and base yaw (1). All axes start together, and the firmware prints `Complete` when every motor has stopped. Base-yaw targets beyond $\pm 1$ revolution are rejected. Each field is parsed into a 16-bit `int`, so a single command can carry at most $\pm 32767$ steps per axis. |
| `l<deg>` | Head servo to `deg`$\in [0, 180]$; 90 is aligned with the arm. |
| `pickup` | Energize both magnets with attract polarity. |
| `place` | Reverse magnet polarity for 5 s, then switch off (only valid after `pickup`). This is a blocking `delay`: steppers and serial are frozen meanwhile. |
| `home` | Stop all motors and reset the step counters to zero. There are no limit switches. |
| `accel_on` / `accel_off` | Enable or disable acceleration ramps. In practice `accel_off` has no effect, because AccelStepper ignores a zero acceleration. |
| `status` | Report step counters, servo angle and magnet state. |
| `<n>c`, `<n>cc`, `<n>m<steps>`, `<n>s<rpm>`, `stop<n>` | Per-motor bring-up commands: one revolution either way, relative steps, speed, stop. |
| `sync34r`, `sync34m<steps>`, `sweep`, `stopall`, `test` | Synchronized X-motor test, servo sweep, emergency stop, self-test. |

# B Hyperparameters

Table 13: Configuration of the main run (`runs/robot_rapcf_clr4_main/config.json`); physics thresholds from `configs/sim/sim.yaml`. Training stopped at its wall-clock budget after about $2 \cdot 10^7$ of the configured $6 \cdot 10^7$ steps.

| Environment and masks | | Network and PPO | |
|---|---|---|---|
| Grid resolution $\Delta$ | 0.5 cm | Embedding dim. $d$ / heads / blocks | 128 / 4 / 4 |
| Container $X \times Y \times Z$ | $60 \times 40 \times 30$ cells | FF expansion / dropout | 2 / 0 |
| Lookahead items | 1 | View encoder | 3 conv, 32 ch, stride 2 |
| Candidate slots $K$ | 32 | Learning rate | $3{\cdot}10^{-4} \rightarrow 3{\cdot}10^{-5}$ |
| Min. support ratio | 0.5 (+ corner/hull) | Clip / value clip | 0.2 / 0.2 |
| Lateral clearance $\lambda$ | 4 mm per side | GAE $\lambda$ / $\gamma$ | 0.95 / 1.0 |
| Carry clearance $c$ | 5 mm | Entropy coef. | $0.01 \rightarrow 0.001$ |
| Envelope $(e_y, e_z)$ | (30, 25) mm | Value coef. / grad. clip | 0.5 / 0.5 |
| DSER $(\alpha, \beta, \gamma, \delta)$ | (1, .05, .05, .01) | Rollout / minibatch | 16 / 2048 |
| Parallel envs $B$ | 2048 | Epochs per update | 4 |
| Physics: disp. / tilt / prev. | 10 mm / 5° / 5 mm | Steps run / wall clock | $1.99 \cdot 10^7$ / 57 min |

# C Interfaces and reproducibility

All modules exchange plain files whose schemas are specified in `docs/interfaces.md`:

- `*.plan.json`: container size, and per item its dimensions, rotation flag, target corner in frame $C$ and pick-row slot. Produced by the policy or by a heuristic.
- `*.traj.json`: labeled segments with synchronized trapezoidal profiles, dense joint samples every 10 ms, and magnet events.
- `*.cmd.txt`: the firmware command stream (Table 12).
- `*.simlog.npz`: time, joint states, box poses and magnet state from twin runs.

Geometry, limits and physical parameters are read only from `overpack/config/robot.yaml`, and step conversion only from `overpack/config/calibration.yaml`. The scripts that produce every table and figure are listed in `paper/figures/README.md`.